this in arrow functions

Same object, two different answers

this.js

```
1 const user = {
2   username: "Aman",
3   hi() { console.log(this.username); },
4   bye: () => console.log(this.username),
5 };
6 user.hi(); // "Aman"
7 user.bye(); // undefined
```

output "Aman" undefined

Why it happens

01 Normal function

Gets this from HOW it is called. user.hi() means this = user.

02 Arrow function

Has NO own this. It takes this from where it was written.

03 Object braces are not a scope

The arrow was written in the outer scope, not "inside" user.

04 Outer this

In a normal browser script, that is window, which has no username.

@specsycoder

Where arrows win

Callbacks inside methods

where-arrows-shine.js

```
const timer = {
  count: 0,
  start() {
    setInterval(() => this.count++, 1000);
  },
};
// arrow keeps this = timer inside start()
```

how it is called	this is
`user.hi()`	**user**
`const f = user.hi; f()`	**undefined / window**
`arrow function`	this of the outer scope
`new Person()`	the new object
`fn.call(obj)`	obj (not for arrows)

Traps to mention

- **call, apply and bind cannot change** an arrow function's this.
- **In ES modules, the outer this is undefined,** so this.username throws a TypeError.
- **Arrows cannot be used with new.** They have no own this.

Your 20-second answer

"A normal function gets this from how it is called, so user.hi() gives user. An arrow function has no own this. It takes this from the scope where it was written, which here is outside the object, so this.username is undefined. I use normal methods on objects and arrows for callbacks inside them."